

HCR_ENGINE - Technical Documentation

Lead Engineer: Masab Rehman | Data Scientist / ML Engineer

Date: August 29, 2026

[LinkedIn Profile](#)

[GitHub Repository](#)

[Personal Portfolio](#)

1. Executive Summary

HCR_ENGINE represents a state-of-the-art Deep Learning pipeline for Handwritten Character and Sequence Recognition. The platform completely automates the digitization of cursive and disconnected handwriting, pushing past the limitations of traditional Optical Character Recognition (OCR).

The system pairs an advanced PyTorch backend with a strikingly designed Brutalist interface for real-time human-in-the-loop testing.

2. Data Processing Pipeline

Handwriting is highly variable in stroke weight, slant, and dimension. The preprocessing pipeline standardizes this data perfectly for neural network consumption.

- Binarization: Applied Otsu's thresholding to isolate ink pixels from complex backgrounds.
- Geometric Standardization: Scaled all images to a fixed height (e.g., 64px) while maintaining the aspect ratio, subsequently padding the right side to a maximum uniform width.
- Segmentation: Utilized horizontal and vertical projection profiles for intelligent line and word segmentation.

3. Neural Network Architecture (CRNN + CTC)

The model architecture abandons naive character-cropping CNNs in favor of a full sequence-to-sequence methodology.

- Convolutional Layers (CNN): Extracts spatial feature maps from the input image. Max-pooling is configured to compress height down to 1 pixel while preserving sequential width.
- Recurrent Layers (BiLSTM): A Bidirectional LSTM processes the sequential slices, capturing linguistic context and cursive connections.
- Connectionist Temporal Classification (CTC): The CTC loss function allows the network to map variable-length input sequences to variable-length text outputs without requiring perfectly aligned training data.

HCR_ENGINE - Technical Documentation

Lead Engineer: Masab Rehman | Data Scientist / ML Engineer

Date: August 29, 2026

4. Model Decoding & Evaluation

Predictions are decoded using Greedy Search (collapsing repeated characters and removing blank tokens).

- Generates character-level confidence scores and Top-K candidate probabilities.
- Evaluated using Character Error Rate (CER) and Word Error Rate (WER) metrics.

5. Brutalist UI & System Integration

The frontend is a masterpiece of functional design, built specifically for diagnostic ML testing.

- Live Drawing Canvas: An HTML5 Canvas allows users to draw characters. On 'mouseup', the drawing is instantly encoded to Base64 and shipped to the Flask API.
- Visual Diagnostics: The UI dynamically renders the pre-processed tensor (how the AI sees the image) alongside the prediction and confidence gauges.
- Real-Time API: Zero-latency endpoints ensure the UI feels like a seamless desktop application.